

Unified Mathematical Model

Project: Smart Combustion Testbed

Document: Mathematical Model

Version: 0.1

Status: Initial architecture

Authors: Lanzapapas-lab

1. Purpose

This document defines the mathematical model used to represent the dynamic state of the experimental system.

The objective is to establish a unified mathematical framework connecting:

1. Physical dynamics
2. Thermodynamic variables
3. Motion and orientation
4. Sensor measurements
5. State estimation
6. Telemetry
7. Numerical simulation
8. Experimental data analysis

The central representation is a continuous-time nonlinear state-space model:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, \boldsymbol{\theta}, t) + \mathbf{w}$$

with measurement model:

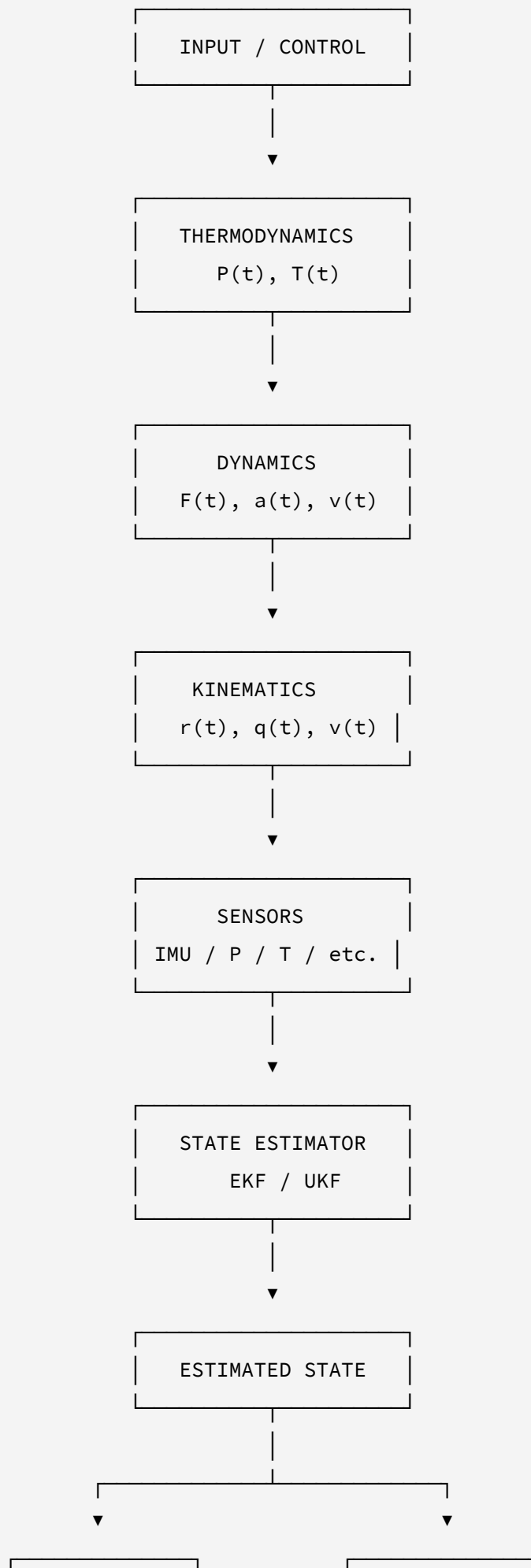
$$\mathbf{y} = \mathbf{h}(\mathbf{x}, \mathbf{u}, \boldsymbol{\theta}, t) + \mathbf{v}$$

where:

- $\mathbf{x}$ is the physical state.
 - $\mathbf{u}$ is the input vector.
 - $\boldsymbol{\theta}$ contains model parameters.
 - $\mathbf{y}$ contains sensor measurements.
 - $\mathbf{w}$ represents process uncertainty.
 - $\mathbf{v}$ represents measurement noise.
-

2. Model Philosophy

The model is intentionally divided into physical subsystems internally, while exposing a single global state-space representation.



The important design principle is:

Simulation, estimation and telemetry must use the same definition of state and coordinate systems.

3. Coordinate Systems

Three reference frames are defined.

3.1 World frame

The world frame is an inertial or approximately inertial reference frame.

It is denoted:

$\mathcal{W}$

with coordinates:

$$\mathbf{r} = [x \ y \ z]$$

where:

- x : horizontal coordinate
- y : horizontal coordinate
- z : vertical coordinate

The gravitational acceleration is represented by:

$$\mathbf{g} = [0 \ 0 \ -g]$$

3.2 Body frame

The body-fixed frame is attached to the experimental vehicle.

It is denoted:

$\mathcal{B}$

The IMU measurements are normally expressed in this frame.

Body-frame quantities are indicated using the subscript B .

For example:

$$\mathbf{a}_B = [a_x \ a_y \ a_z]$$

3.3 Sensor frame

The physical sensor coordinate system may differ from the body frame.

Therefore:

$$\mathcal{S} \neq \mathcal{B}$$

may occur.

A calibration transformation must therefore be defined:

$$\mathbf{a}_B = R_{BS}\mathbf{a}_S$$

where:

$$R_{BS} \in SO(3)$$

is the rotation from sensor coordinates to body coordinates.

4. State Vector

The complete state is defined as:

$$\mathbf{x} = [\mathbf{r} \ \mathbf{v} \ \mathbf{q} \ \mathbf{b}_a \ \mathbf{b}_g \ P \ T]$$

Expanded:

$$\mathbf{x} = [x \ y \ z \ v_x \ v_y \ v_z \ q_0 \ q_1 \ q_2 \ q_3 \ b_{ax} \ b_{ay} \ b_{az} \ b_{gx} \ b_{gy} \ b_{gz} \ P \ T]^T$$

The nominal dimension is therefore:

$$n_x = 18$$

5. State Variables

5.1 Position

$$\mathbf{r} = [x \ \color{red}{y} \ \color{red}{z}]$$

Units:

$[m]$

5.2 Velocity

$$\mathbf{v} = [v_x \setminus \mathbf{v}_y \setminus \mathbf{v}_z]$$

Units:

$[m/s]$

5.3 Orientation

Orientation is represented using a quaternion:

$$\mathbf{q} = [q_0 \setminus \mathbf{q}_1 \setminus \mathbf{q}_2 \setminus \mathbf{q}_3]$$

with normalization condition:

$$|\mathbf{q}| = 1$$

Quaternions are used instead of Euler angles to avoid singularities associated with pitch/roll representations.

5.4 Accelerometer bias

$$\mathbf{b}_a = [b_{ax} \setminus \mathbf{b}_{ay} \setminus \mathbf{b}_{az}]$$

Units:

$[m/s^2]$

The bias is considered a slowly varying quantity.

5.5 Gyroscope bias

$$\mathbf{b}_g = [b_{gx} \setminus \mathbf{b}_{gy} \setminus \mathbf{b}_{gz}]$$

Units:

$[rad/s]$

5.6 Pressure

P

Units:

$[Pa]$

The pressure state may represent the pressure associated with the monitored experimental subsystem.

5.7 Temperature

T

Units:

$[K]$

All thermodynamic calculations should use Kelvin internally.

6. Input Vector

The general input vector is:

$$\mathbf{u}(t) = [\mathbf{u}_c \ \mathbf{u}_e]$$

where:

- $\mathbf{u}_c$: controlled inputs
- $\mathbf{u}_e$: experimental/environmental inputs

The exact contents depend on the experiment.

Examples include:

- commanded actuator state
- external environmental conditions
- known experimental events
- known configuration states

The model must distinguish between:

measured input

and:

estimated state

7. Parameter Vector

The model parameters are represented by:

$$[\theta_1, \theta_2, \dots, \theta_p]^T$$

Typical categories are:

Physical parameters

- m : system mass
- geometric quantities
- inertial properties
- reference areas
- aerodynamic coefficients

Sensor parameters

- accelerometer bias
- gyroscope bias
- sensor scale factors
- sensor misalignment
- noise parameters

Thermodynamic parameters

- gas constants
- heat-capacity ratios
- thermal parameters

Environmental parameters

- gravitational acceleration
- atmospheric conditions
- ambient temperature
- ambient pressure

The exact numerical values are configuration data rather than hard-coded constants.

8. Kinematic Model

Position evolves according to:

$$\dot{\mathbf{r}} = \mathbf{v}$$

or:

$$[v_x \ v_y \ v_z]$$

9. Dynamic Model

Newton's second law is used:

$$\sum_i \mathbf{F}_i$$

Therefore:

$$\frac{1}{m} \sum_i \mathbf{F}_i$$

The total force is conceptually decomposed as:

$$\mathbf{F}_{\text{gravity}} + \mathbf{F}_{\text{environment}} + \mathbf{F}_{\text{actuation}} + \mathbf{F}_{\text{other}}$$

The implementation must keep these contributions separate.

This allows individual terms to be enabled or disabled during validation.

10. Gravity

Gravity is represented in the world frame as:

$$\mathbf{F}_g = m\mathbf{g}$$

with:

$$\mathbf{g} = [0 \ 0 \ -g]$$

Therefore:

$$\mathbf{g} + \frac{\mathbf{F}_{\text{other}}}{m}$$

11. Orientation Dynamics

Let:

$$[\omega_x \ \omega_y \ \omega_z]$$

be angular velocity.

Define:

$$\Omega(\omega) = \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z & \omega_x & 0 & \omega_z & -\omega_y & \omega_y & -\omega_z & 0 & \omega_x & \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix}$$

The quaternion dynamics are:

$$\frac{1}{2}\Omega(\omega)\mathbf{q}$$

After numerical integration:

$$\mathbf{q} \leftarrow \frac{\mathbf{q}}{|\mathbf{q}|}$$

to maintain the unit-quaternion constraint.

12. Thermodynamic Model

The thermodynamic subsystem provides the evolution of pressure and temperature.

For an ideal gas:

$$PV = nRT$$

where:

- P : pressure
- V : volume
- n : amount of substance
- R : universal gas constant
- T : absolute temperature

For a simplified adiabatic ideal-gas process:

$$PV^\gamma = C$$

where:

$$\gamma = \frac{C_p}{C_v}$$

This equation is used as a model assumption rather than as a universal representation of the real experimental process.

13. Thermodynamic State Equations

The generic thermodynamic subsystem is represented by:

$$\dot{P} = f_P(P, T, \mathbf{u}, \boldsymbol{\theta}, t)$$

$$\dot{T} = f_T(P, T, \mathbf{u}, \boldsymbol{\theta}, t)$$

The precise functions are deliberately implemented as independent modules.

This allows different models to be tested:

```
ThermodynamicModel
├─ IdealGasModel
├─ SimplifiedAdiabaticModel
├─ ExperimentalModel
└─ FutureModel
```

The rest of the state-space architecture remains unchanged.

14. Coupling Between Thermodynamics and Dynamics

The important connection is:

$$P, T \rightarrow \mathbf{F} \rightarrow \mathbf{a} \rightarrow \mathbf{v} \rightarrow \mathbf{r}$$

The complete chain is:

$$(P, T) \rightarrow \mathbf{F}_{\text{physical}} \rightarrow \dot{\mathbf{v}} \rightarrow \mathbf{v} \rightarrow \mathbf{r}$$

This is the central coupling of the model.

15. Aerodynamic Model

The aerodynamic subsystem represents environmental forces.

In general:

$$f_{\text{aero}}(\mathbf{v}, \boldsymbol{\omega}, \rho, \boldsymbol{\theta}_{\text{aero}})$$

For a simplified drag model:

$$F_D = \frac{1}{2} \rho C_D A v^2$$

where:

- ρ : air density
- C_D : drag coefficient
- A : reference area
- $v = |\mathbf{v}|$: speed

The force vector acts opposite to the relative airflow direction.

For the initial implementation, aerodynamic coefficients should be treated as experimentally determined or literature-derived parameters rather than optimization variables.

16. Relative Air Velocity

If wind is included:

$$\mathbf{v} - \mathbf{v}_{\text{wind}}$$

and:

$$|\mathbf{v}_{\text{rel}}|$$

The aerodynamic subsystem should operate on relative velocity rather than absolute ground velocity.

17. Atmospheric Model

A basic atmospheric model can provide:

$$\rho = \rho(z, T, P)$$

For the initial implementation, constant atmospheric conditions may be assumed.

A more advanced implementation can use altitude-dependent atmospheric properties.

The architecture therefore becomes:

```
EnvironmentModel
├── ConstantEnvironment
└── AtmosphereModel
```

18. Complete Continuous-Time Model

The complete state derivative is:

$$[\dot{\mathbf{r}} \ \dot{\mathbf{v}} \ \dot{\mathbf{q}} \ \dot{\mathbf{b}}_a \ \dot{\mathbf{b}}_g \ \dot{P} \ \dot{T}]$$

with:

$$\begin{aligned}\dot{\mathbf{r}} &= \mathbf{v} \\ \frac{\mathbf{F}_g + \mathbf{F}_{\text{aero}} + \mathbf{F}_{\text{actuation}} + \mathbf{F}_{\text{other}}}{m} \\ \frac{1}{2}\Omega(\boldsymbol{\omega})\mathbf{q} \\ \dot{\mathbf{b}}_a &= \mathbf{w}_{ba} \\ \dot{\mathbf{b}}_g &= \mathbf{w}_{bg} \\ \dot{P} &= f_P(P, T, \mathbf{u}, \boldsymbol{\theta}, t) \\ \dot{T} &= f_T(P, T, \mathbf{u}, \boldsymbol{\theta}, t)\end{aligned}$$

Therefore:

$$\mathbf{f}(\mathbf{x}, \mathbf{u}, \boldsymbol{\theta}, t) + \mathbf{w}$$

19. Sensor Model

The physical state is not directly observable.

Instead, sensors provide measurements:

$$\boxed{\mathbf{y} = \mathbf{h}(\mathbf{x}, \mathbf{u}, \boldsymbol{\theta}) + \mathbf{v}}$$

The initial sensor set is:

IMU

- └─ Accelerometer
- └─ Gyroscope

Environmental / Experimental

- └─ Pressure
- └─ Temperature

Optional:

External camera
GNSS
Magnetometer
Additional event sensors

20. Accelerometer Model

The accelerometer measures specific force rather than simply world-frame acceleration.

Let:

$R(\mathbf{q})$

be the rotation matrix associated with the quaternion.

The ideal accelerometer measurement can be represented as:

$$R(\mathbf{q})^T (\dot{\mathbf{v}} - \mathbf{g})$$

- $\mathbf{b}_a$
 - $\mathbf{n}_a$
- }
\$\$

where:

- $\mathbf{a}_m$: measured acceleration
- $\dot{\mathbf{v}}$: world-frame acceleration
- $\mathbf{g}$: gravitational acceleration
- $\mathbf{b}_a$: accelerometer bias
- $\mathbf{n}_a$: accelerometer noise

21. Gyroscope Model

The gyroscope measurement is:

$$\boldsymbol{\omega}$$

- $\mathbf{b}_g$

- $\mathbf{n}_g$

where:

- ω_m : measured angular velocity
 - ω : true angular velocity
 - $\mathbf{b}_g$: bias
 - $\mathbf{n}_g$: measurement noise
-

22. Pressure Sensor Model

$$P_m = P + \eta_P$$

where:

$$\eta_P \sim \mathcal{N}(0, \sigma_P^2)$$

is the pressure measurement noise.

23. Temperature Sensor Model

$$T_m = T + \eta_T$$

where:

$$\eta_T \sim \mathcal{N}(0, \sigma_T^2)$$

24. Sensor Bias Model

Biases are not assumed perfectly constant.

A simple random-walk model is:

$$\dot{\mathbf{b}}_a = \mathbf{w}_{ba}$$

$$\dot{\mathbf{b}}_g = \mathbf{w}_{bg}$$

where:

$$\mathbf{w}_{ba} \sim \mathcal{N}(0, Q_{ba})$$

and:

$$\mathbf{w}_{bg} \sim \mathcal{N}(0, Q_{bg})$$

This allows the estimator to adapt to slowly changing sensor offsets.

25. Noise Model

Process noise:

$$\mathbf{w} \sim \mathcal{N}(0, Q)$$

Measurement noise:

$$\mathbf{v} \sim \mathcal{N}(0, R)$$

where:

$$Q = \text{cov}(\mathbf{w})$$

and:

$$R = \text{cov}(\mathbf{v})$$

The matrices Q and R are experimentally estimated whenever possible.

26. Discrete-Time Model

The ESP32 and computer operate in discrete time.

For a sampling interval:

$$\Delta t = t_k - t_{k-1}$$

the continuous model becomes:

$$f_d(\mathbf{x}_{k-1}, \mathbf{u}_k, \boldsymbol{\theta}, \Delta t) + \mathbf{w}_k$$

and:

$$h(\mathbf{x}_k) + \mathbf{v}_k$$

The first implementation should use numerical integration rather than manually deriving every discrete equation.

27. Numerical Integration

The initial implementation should support:

Euler

$$x_{k+1} = x_k + \dot{x}_k \Delta t$$

Simple but relatively inaccurate.

Runge-Kutta

The recommended initial simulation integrator is:

RK4

because it provides substantially better numerical accuracy for many smooth nonlinear systems while remaining computationally manageable.

For real-time embedded operation, a simpler integration scheme may later be implemented.

28. State Estimation

The estimator receives:

```
MODEL PREDICTION
+
SENSOR MEASUREMENTS
↓
STATE ESTIMATOR
↓
BEST ESTIMATE OF x
```

The initial estimator will be an Extended Kalman Filter.

29. Extended Kalman Filter

The nonlinear model is:

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k) + \mathbf{w}_k$$

$$\mathbf{y}_k = h(\mathbf{x}_k) + \mathbf{v}_k$$

29.1 Prediction

$$f(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k)$$

The covariance is:

$$F_k P_{k-1} F_k^T + Q_k$$

where:

$$F_k = \frac{\partial f}{\partial x}$$

is the state-transition Jacobian.

29.2 Measurement prediction

$$h(\hat{\mathbf{x}}_{k|k-1})$$

Innovation:

$$\mathbf{y}_k - \hat{\mathbf{y}}_k$$

29.3 Measurement Jacobian

$$H_k = \frac{\partial h}{\partial x}$$

29.4 Innovation covariance

$$S_k = H_k P_{k|k-1} H_k^T + R_k$$

29.5 Kalman gain

$$K_k = P_{k|k-1} H_k^T S_k^{-1}$$

29.6 State correction

$$\hat{\mathbf{x}}_{k|k-1} + K_k \mathbf{r}_k$$

29.7 Covariance correction

$$P_k = (I - K_k H_k) P_{k|k-1}$$

For numerical robustness, the Joseph form should be considered:

$$P_k = (I - KH)P(I - KH)^T + K R K^T$$

30. Quaternion Considerations in the EKF

A quaternion contains four variables but only three independent orientation degrees of freedom.

Therefore:

$$q_0^2 + q_1^2 + q_2^2 + q_3^2 = 1$$

must always hold.

The implementation should normalize the quaternion after each update.

For a future high-quality implementation, an Error-State EKF can be introduced to represent orientation errors with a minimal three-dimensional perturbation.

31. Observability

Not every state can necessarily be recovered from every sensor configuration.

For example:

- Position cannot generally be obtained accurately from an IMU alone for long periods.
- Velocity obtained by integrating acceleration accumulates error.
- Gyroscope bias produces orientation drift.
- Pressure can provide useful information about vertical/environmental state.
- External vision can provide additional position information.

Therefore the sensor architecture should be considered part of the mathematical model.

The project should explicitly study:

Which states are observable from which sensors?

32. Camera Measurement Model

If an external camera is used, image measurements can be represented as:

$$h_{\text{vision}}(\mathbf{x}) + \mathbf{v}_{\text{vision}}$$

Possible outputs include:

- image position
- apparent velocity
- orientation
- trajectory points

The vision system should initially be treated as an independent measurement source.

Later:

$$\boxed{\text{IMU} + \text{Vision} \rightarrow \text{Sensor Fusion}}$$

can be implemented.

33. Model Error

The real system will never perfectly match the model.

Define:

$$\mathbf{y}_{\text{real}}(t)$$

$$\mathbf{y}_{\text{model}}(t)$$

The error should be analysed statistically.

Examples:

$$RMSE = \sqrt{\frac{1}{N} \sum_{k=1}^N e_k^2}$$

Mean error:

$$\mu_e = \frac{1}{N} \sum_{k=1}^N e_k$$

Standard deviation:

$$\sigma_e = \sqrt{\frac{1}{N-1} \sum_{k=1}^N (e_k - \mu_e)^2}$$

34. Parameter Identification

Model parameters can be refined using experimental measurements.

Define:

$$[\theta_1, \theta_2, \dots, \theta_p]^T$$

and an objective function:

$$\sum_{k=1}^N \mathbf{y}_k - \hat{\mathbf{y}}_k(\boldsymbol{\theta})^2$$

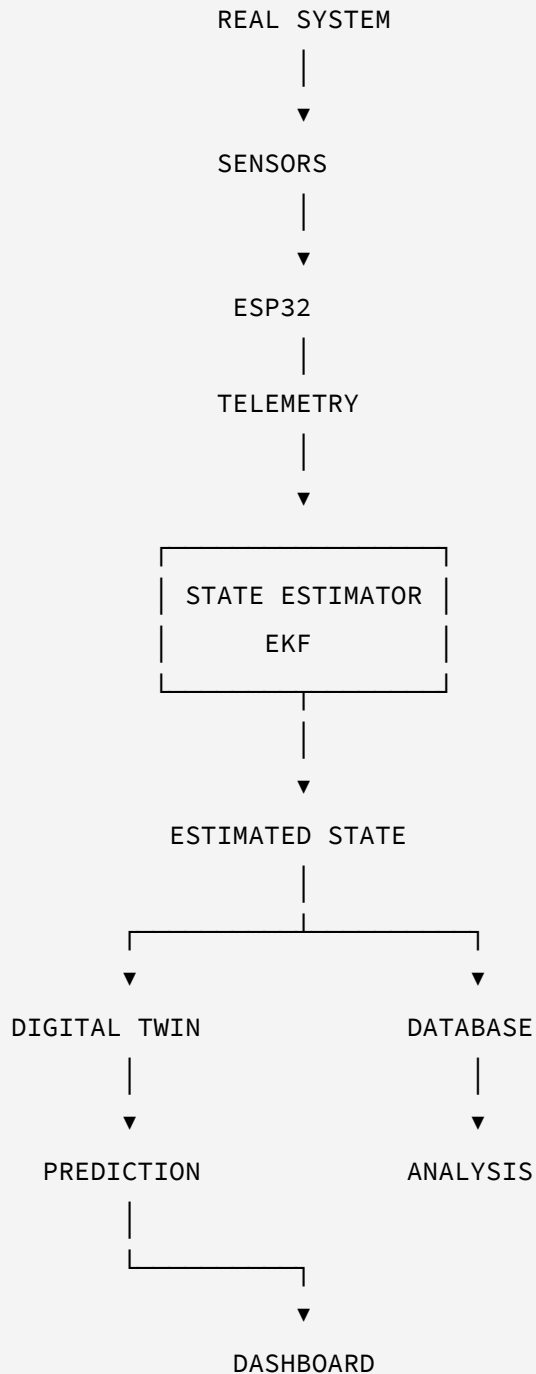
The identification problem becomes:

$$\arg \min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

This is useful for estimating uncertain properties from controlled experiments.

35. Digital Twin

The model acts as a digital twin when it receives real measurements and continuously estimates the system state.



The digital twin therefore maintains:

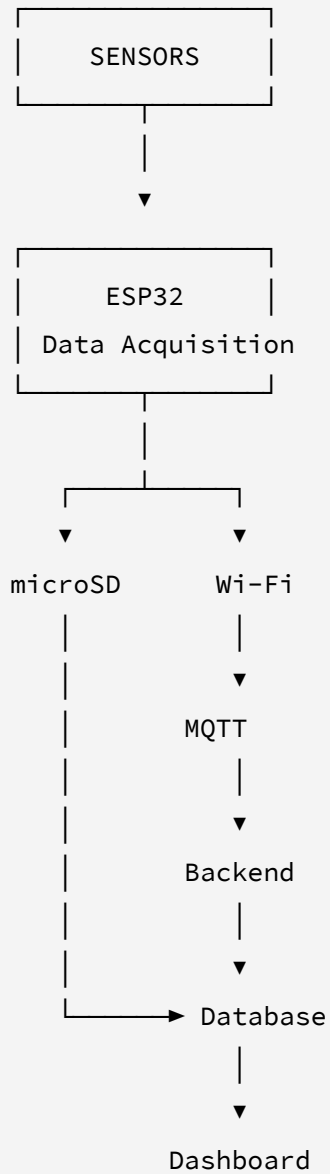
$$\hat{\mathbf{x}}(t)$$

rather than merely displaying raw sensor values.

36. Telemetry Architecture

The mathematical model should not depend directly on the communications protocol.

The architecture is:



The complete raw dataset should preferably be stored locally so that temporary communication loss does not destroy experimental data.

37. Telemetry Data Structure

A measurement record should contain at minimum:

```
{
  "timestamp": 0.000,
  "ax": 0.0,
  "ay": 0.0,
  "az": 0.0,
  "gx": 0.0,
  "gy": 0.0,
  "gz": 0.0,
  "pressure": 0.0,
  "temperature": 0.0
}
```

An estimated-state record may contain:

```
{
  "timestamp": 0.000,
  "x": 0.0,
  "y": 0.0,
  "z": 0.0,
  "vx": 0.0,
  "vy": 0.0,
  "vz": 0.0,
  "q0": 1.0,
  "q1": 0.0,
  "q2": 0.0,
  "q3": 0.0,
  "pressure": 0.0,
  "temperature": 0.0
}
```

Units must be explicitly documented.

38. Time Synchronization

Time is fundamental to the model.

Every measurement must have:

$$t_k$$

The system should use a monotonic local clock for relative timing.

All measurements should be associated with a common timeline:

$$t_0, t_1, \dots, t_N$$

Sampling intervals are:

$$\Delta t_k = t_k - t_{k-1}$$

The model should not assume that every sample arrives at exactly the nominal sampling interval.

39. Sampling

Different sensors may operate at different frequencies.

For example:

```
IMU           → high rate
Pressure      → medium rate
Temperature   → low rate
Camera        → independent rate
Telemetry     → independent rate
```

Therefore the architecture must support asynchronous measurements.

The EKF can process a measurement whenever it arrives:

```
prediction → IMU
prediction → IMU
prediction → IMU
measurement update → pressure
prediction → IMU
measurement update → vision
...
```

40. Initial Assumptions

Version 0.1 uses the following assumptions:

1. The system can be approximated as a rigid body.
2. The reference frame is approximately inertial.

3. Gravity is known.
 4. Sensors are calibrated before experiments.
 5. Sensor noise can initially be approximated as Gaussian.
 6. Accelerometer and gyroscope biases vary slowly.
 7. Quaternion orientation remains normalized.
 8. The thermodynamic subsystem can initially be represented using a simplified model.
 9. Aerodynamic parameters are treated as model parameters.
 10. Environmental conditions are either measured or specified.
 11. Communication does not alter the physical state.
 12. Telemetry loss does not invalidate locally stored measurements.
 13. The model is validated experimentally before increasing its complexity.
-

41. Model Hierarchy

The model should evolve progressively.

Level 0 — Kinematic

$$\dot{\mathbf{r}} = \mathbf{v}$$

Used to test the software architecture.

Level 1 — Basic dynamics

$$m\dot{\mathbf{v}} = \sum \mathbf{F}$$

Gravity is included.

Level 2 — Orientation

Quaternion dynamics are added.

Level 3 — Sensor model

IMU, pressure and temperature are simulated.

Level 4 — Noise and bias

$$\mathbf{b}_a, \mathbf{b}_g, \mathbf{w}, \mathbf{v}$$

are introduced.

Level 5 — EKF

The estimated state is compared against the simulated ground truth.

Level 6 — Real telemetry

The simulated sensor stream is replaced by ESP32 measurements.

Level 7 — Experimental parameter identification

Unknown parameters are estimated from real experiments.

Level 8 — Digital twin

Simulation and real-time estimation operate simultaneously.

42. Python Architecture

The Python implementation should mirror the mathematical architecture.

```
software/
└─ mathematical_model/
    ├── __init__.py
    |
    ├── state.py
    ├── parameters.py
    ├── dynamics.py
    ├── kinematics.py
    ├── thermodynamics.py
    ├── aerodynamics.py
    ├── environment.py
    ├── sensors.py
    ├── noise.py
    ├── integrators.py
    ├── ekf.py
    ├── simulation.py
    ├── telemetry.py
    ├── validation.py
    └─ visualization.py
```

43. **state.py**

Responsible only for representing the system state.

Conceptually:

```
@dataclass
class State:
    position: np.ndarray
    velocity: np.ndarray
    quaternion: np.ndarray
    accel_bias: np.ndarray
    gyro_bias: np.ndarray
    pressure: float
    temperature: float
```

The class should also provide:

```
normalize_quaternion()  
as_vector()  
from_vector()
```

44. **parameters.py**

Contains physical and sensor parameters.

```
@dataclass  
class ModelParameters:  
    mass: float  
    gravity: float  
  
    accel_noise_std: float  
    gyro_noise_std: float  
  
    pressure_noise_std: float  
    temperature_noise_std: float  
  
    accel_bias_noise: float  
    gyro_bias_noise: float
```

Parameters must not be scattered through the source code.

45. **kinematics.py**

Responsible for:

- position integration
- quaternion propagation
- coordinate transformations
- rotation matrices

Main conceptual functions:

```
position_derivative(...)
quaternion_derivative(...)
quaternion_to_rotation(...)
rotation_to_quaternion(...)
```

46. **dynamics.py**

Responsible for calculating forces and acceleration.

Conceptual API:

```
def compute_forces(state, inputs, params):
    ...

def acceleration(state, inputs, params):
    ...
```

The force calculation should return individual contributions:

```
{
    "gravity": ...,
    "aerodynamic": ...,
    "actuation": ...,
    "other": ...
}
```

This makes debugging and validation much easier.

47. **thermodynamics.py**

Responsible for:

$P(t), T(t)$

Conceptual interface:

```
class ThermodynamicModel:

    def derivatives(self, state, inputs, params):
        ...

    def pressure(self, state, inputs, params):
        ...

    def temperature(self, state, inputs, params):
        ...
```

Different thermodynamic models can inherit from this interface.

48. **aerodynamics.py**

Responsible for environmental force models.

Conceptual interface:

```
class AerodynamicModel:

    def force(self, state, environment, params):
        ...
```

The module must remain independent of the telemetry layer.

49. **sensors.py**

Simulates the sensor measurements.

```
def accelerometer_measurement(state, params):  
    ...  
  
def gyroscope_measurement(state, params):  
    ...  
  
def pressure_measurement(state, params):  
    ...  
  
def temperature_measurement(state, params):  
    ...
```

This is extremely useful because it allows us to test the EKF before connecting physical hardware.

50. `noise.py`

Contains stochastic models.

```
def gaussian_noise(covariance):  
    ...  
  
def random_walk(previous_bias, dt, covariance):  
    ...
```

A fixed random seed should be supported for reproducible simulations.

51. `integrators.py`

Contains:

```
euler_step(...)  
rk4_step(...)
```

The simulation engine should be independent of the physical model.

52. `simulation.py`

This module combines everything.

Conceptually:

```
state = initial_state()

for t in time:

    derivative = model.derivatives(
        state,
        inputs,
        parameters
    )

    state = integrator.step(
        state,
        derivative,
        dt
    )

    measurements = sensor_model.measure(
        state
    )

    save(state, measurements)
```

The important distinction is:

```
TRUE STATE
  ↓
PHYSICAL MODEL
  ↓
SENSOR MODEL
  ↓
MEASUREMENTS
```

This allows us to test whether the estimator can recover the known simulated state.

53. ekf.py

The estimator should expose:

```
class ExtendedKalmanFilter:

    def predict(self, state, covariance, inputs, dt):
        ...

    def update(self, measurement):
        ...

    def step(self, measurement, inputs, dt):
        ...
```

The EKF should never contain hardware-specific code.

54. telemetry.py

Responsible only for communication and data serialization.

Possible interfaces:

```
publish_measurement(...)
publish_estimated_state(...)
publish_system_status(...)
```

The mathematical model should not know whether the data arrived through:

- CSV
 - MQTT
 - serial
 - Wi-Fi
 - microSD
-

55. validation.py

This module compares:

MODEL
vs
EXPERIMENT

It should calculate:

- RMSE
- mean error
- standard deviation
- maximum error
- correlation
- residuals

Example conceptual interface:

```
compare_predictions(  
    model_data,  
    experimental_data  
)
```

56. **visualization.py**

Should generate:

- position vs time
- velocity vs time
- acceleration vs time
- angular velocity
- pressure
- temperature
- model vs measurement
- residuals
- estimated vs true state

The plotting system should never modify the underlying data.

57. Recommended Python Stack

Core:

```
Python 3.11+  
NumPy  
SciPy  
Pandas  
Matplotlib
```

Symbolic mathematics:

```
SymPy
```

Optional later:

```
FilterPy  
JAX  
PyTorch  
OpenCV  
Plotly
```

The first implementation should rely primarily on:

NumPy + SciPy + SymPy + Matplotlib

rather than introducing machine learning unnecessarily.

58. Role of AI

AI should be used as an engineering copilot, not as the source of physical truth.

Useful tasks:

ChatGPT / Claude

- derive equations
- explain mathematical concepts
- review Python
- generate unit tests
- identify dimensional inconsistencies
- review documentation
- suggest numerical methods

- explain sensor-fusion algorithms

Python

Python remains the computational authority:

- numerical integration
- optimization
- filtering
- statistical analysis
- simulation
- visualization

SymPy

Used to verify:

$$\frac{\partial f}{\partial x}$$

and derive Jacobians.

Experimental data

Used to determine whether the model is actually correct.

The hierarchy should therefore be:

Physics > Experimental data > Numerical implementation > AI assistance

59. Unit System

The complete mathematical model uses SI units.

Quantity	Unit
Position	m
Velocity	m/s
Acceleration	m/s ²
Angular velocity	rad/s
Angle	rad
Mass	kg
Force	N
Pressure	Pa

Quantity	Unit
Temperature	K
Time	s
Density	kg/m ³

Conversion to human-readable units should occur only at the visualization/UI layer.

60. Dimensional Validation

Every equation must pass dimensional analysis.

For example:

$$F = ma$$

gives:

$$[N] = [kg][m/s^2]$$

Similarly:

$$PV = nRT$$

must satisfy:

$$[Pa][m^3] = [mol][J/(mol, K)][K]$$

Dimensional checks should become part of the validation process.

61. Configuration Architecture

Model configuration should be stored separately:

```

config/
├─ default.yaml
├─ simulation.yaml
├─ sensors.yaml
└─ experiments/

```

Example conceptual configuration:

```
simulation:
    timestep: ...
    duration: ...

sensors:
    imu:
        accelerometer:
            noise_std: ...
        gyroscope:
            noise_std: ...

environment:
    gravity: ...
    temperature: ...
    pressure: ...
```

No experimental constants should be hidden inside Python functions.

62. Experiment Data Structure

Each experiment should have its own directory:

```
data/
├── experiments/
│   └── experiment_YYYYMMDD_NNN/
│       ├── raw/
│       ├── processed/
│       ├── metadata.json
│       └── README.md
```

metadata.json should contain:

- experiment ID
- date/time
- firmware version
- model version
- sensor configuration
- calibration version
- environmental conditions

- sampling configuration
- experiment notes

This makes experiments reproducible.

63. Model Versioning

Every dataset should record the mathematical-model version.

For example:

```
model_version = "0.1.0"
```

This prevents a major problem:

comparing data generated with different versions of the model without knowing it.

64. Testing Strategy

The mathematical model should have automated tests.

```
tests/  
├─ test_state.py  
├─ test_quaternion.py  
├─ test_dynamics.py  
├─ test_thermodynamics.py  
├─ test_sensors.py  
├─ test_integrators.py  
└─ test_ekf.py
```

65. Fundamental Tests

The first tests should include:

Quaternion normalization

Verify:

$$|\mathbf{q}| \approx 1$$

Zero-force test

If all external forces except gravity are disabled:

$$\dot{\mathbf{v}} = \mathbf{g}$$

Static sensor test

When stationary, verify the accelerometer produces the expected specific-force measurement.

Zero-angular-rate test

If:

$$\boldsymbol{\omega} = 0$$

then:

$$\dot{\mathbf{q}} = 0$$

Noise test

Verify simulated noise approximately matches the specified covariance.

EKF convergence

Start the estimator with a known state error and verify that:

$$|\hat{\mathbf{x}} - \mathbf{x}_{\text{true}}|$$

decreases under appropriate sensor conditions.

66. Development Roadmap

Phase 1 — Mathematical foundation

Implement:

```
State
Parameters
Coordinate systems
Quaternion mathematics
```

Phase 2 — Basic simulation

Implement:

```
position
velocity
gravity
numerical integration
```

Phase 3 — Orientation

Implement:

```
gyroscope
quaternion propagation
rotation matrices
```

Phase 4 — Sensor simulation

Implement:

```
accelerometer  
gyroscope  
pressure  
temperature  
noise  
bias
```

Phase 5 — EKF

Implement:

```
prediction  
measurement update  
covariance  
Jacobians
```

Phase 6 — Real ESP32 data

Replace:

```
simulated sensors
```

with:

```
real telemetry
```

while keeping exactly the same estimator.

Phase 7 — Experimental validation

Compare:

x_{model}

with:

$x_{\text{experimental}}$

and calculate residuals.

Phase 8 — Parameter identification

Estimate uncertain parameters using experimental data.

Phase 9 — Digital twin

Run:

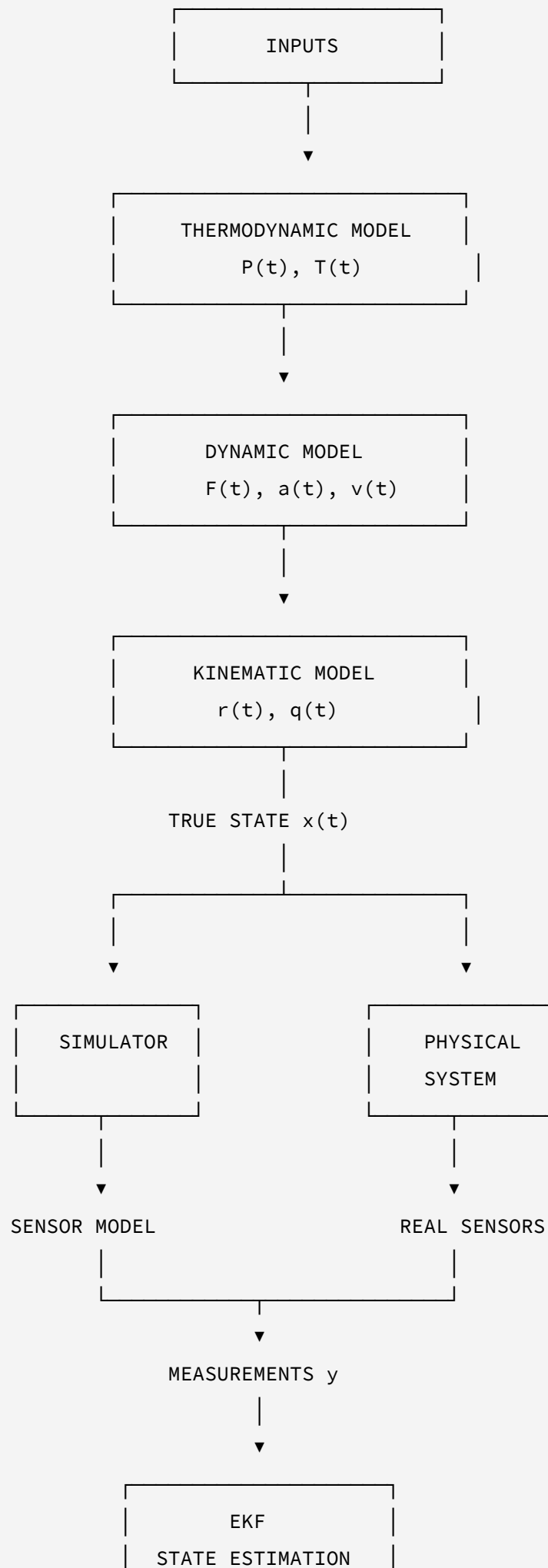
```
graph TD; A[REAL SYSTEM] -- "+" --> B[MATHEMATICAL MODEL]; B -- "+" --> C[EKF]; C -- "+" --> D[DATABASE]; D -- "+" --> E[DASHBOARD];
```

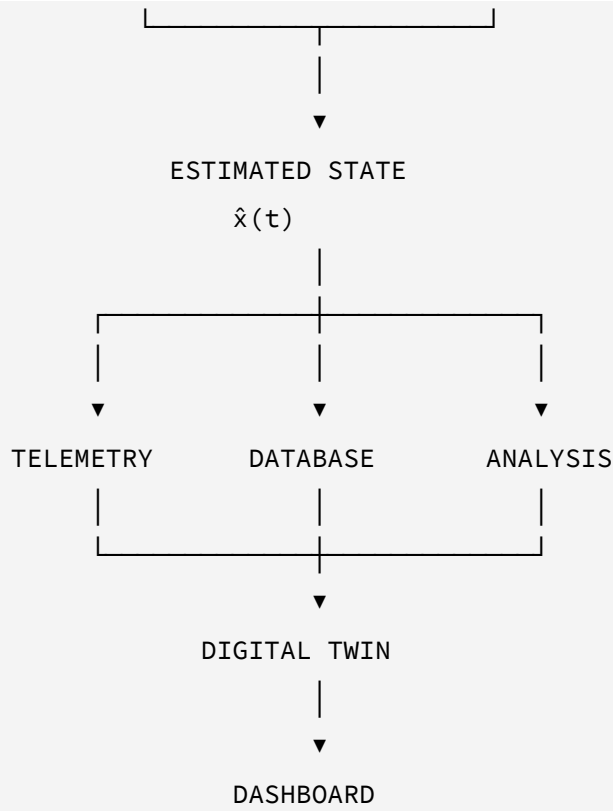
REAL SYSTEM
+
MATHEMATICAL MODEL
+
EKF
+
DATABASE
+
DASHBOARD

in real time.

67. Final Unified Architecture

The complete system can therefore be represented as:





68. Final Mathematical Definition

The entire system is ultimately represented by:

$$\mathbf{x} = [\mathbf{r} \ \mathbf{v} \ \mathbf{q} \ \mathbf{b}_a \ \mathbf{b}_g \ P \ T]$$

with:

$$\mathbf{f}(\mathbf{x}, \mathbf{u}, \boldsymbol{\theta}, t) + \mathbf{w}$$

and:

$$\mathbf{h}(\mathbf{x}, \mathbf{u}, \boldsymbol{\theta}, t) + \mathbf{v}$$

The estimator computes:

$$\text{EKF}(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k, \mathbf{y}_k)$$

and the digital twin maintains:

$$\hat{\mathbf{x}}(t) \approx \mathbf{x}_{\text{real}}(t)$$

subject to model and measurement uncertainty.

This establishes a single mathematical framework connecting:

$$\text{Physics} \rightarrow \text{Dynamics} \rightarrow \text{Sensors} \rightarrow \text{Estimation} \rightarrow \text{Telemetry} \rightarrow \text{Digital Twin}$$

69. Next Implementation Target

The first executable version should **not** implement the complete model at once.

The recommended order is:

```
01  State class
02  Quaternion mathematics
03  Basic dynamics
04  RK4 integrator
05  Simulated IMU
06  Sensor noise/bias
07  EKF
08  Plot true vs estimated state
09  ESP32 telemetry input
10  Real experimental validation
```

The first milestone is therefore:

Simulated system → Simulated sensors → EKF → Recovered state
--

Only after this works should the real ESP32 telemetry be connected.

The model should be treated as a living document. Each new physical subsystem should add or replace a well-defined function inside $\mathbf{f}$ or $\mathbf{h}$, rather than changing the overall architecture.